

Important Concepts of Machine Learning

Dataset :

A **dataset** is a structured collection of data used for analysis and training machine learning models. In ML, each **row** typically represents an example (sample or instance), and each **column** represents a **feature** (input) or **label** (output).

Example (Iris Dataset Sample)

Sepal Length		Sepal Width Petal Length	Petal Width	Species
5.1	3.5	1.4	0.2	Setosa

- **Features:** Sepal Length, Sepal Width, Petal Length, Petal Width
- **Label:** Species

Ways to Load Datasets in Python :

Method 1: Load Built-in Datasets (from sklearn)

```
from sklearn.datasets import load_iris
data = load_iris()
print(data.data)
```

Method 2: Load from a CSV file

```
import pandas as pd
df = pd.read_csv('iris.csv')
print(df.head())
```

Method 3: Load from a URL

```
url = "https://archive.ics.uci.edu/ml/machine-learning-databases/iris/iris.data"
df = pd.read_csv(url)
print(df.head())
```

Features, Labels, Independent & Dependent

Variables Features (Input Variables)

- These are the inputs to the model.
- Also called **independent variables**.

- Example: Age, Height, Weight, Income

Labels (Target Variable)

- This is the output we are trying to predict.
- Also called **dependent variable**.
- Example: Disease (Yes/No), House Price, Species

Train-Test Splitting

We split our dataset into **training** and **testing** parts to evaluate the model's

performance. • **Training Set:** Used to train the model

- **Testing Set:** Used to evaluate performance on unseen data

Typical Split:

- 80% training and 20% testing

Code Example:

```
from sklearn.model_selection import train_test_split

X = df.drop('species', axis=1) # Features
y = df['species'] # Labels

X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2)
```

Confusion Matrix & Related Terms

Used in classification problems to evaluate the performance of the model.

	Predicted: Positive	Predicted: Negative
Actual: Positive	True Positive (TP)	False Negative (FN)
Actual: Negative	False Positive (FP)	True Negative (TN)

Terms:

- **True Positive (TP):** Model correctly predicted Positive
- **True Negative (TN):** Model correctly predicted Negative
- **False Positive (FP):** Model incorrectly predicted Positive
- **False Negative (FN):** Model incorrectly predicted Negative

Accuracy and Other Evaluation Metrics

◆ Accuracy:

Measures overall correctness of the model.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

◆ Precision:

How many of the predicted positives are actually positive?

$$\text{Precision} = \frac{TP}{TP + FP}$$

◆ Recall (Sensitivity):

How many actual positives were correctly predicted?

$$\text{Recall} = \frac{TP}{TP + FN}$$

◆ F1 Score:

Harmonic mean of precision and recall

$$\text{F1 Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Code Example: Confusion Matrix & Accuracy

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, confusion_matrix
```

```
# 1. Load the Iris dataset
```

```
iris = load_iris()
```

```
X = iris.data # Feature data (sepal & petal measurements)
```

```
y = iris.target # Labels (species: 0, 1, 2)
```

```
# 2. Split the dataset (80% train, 20% test)
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
```

```
# 3. Create and train the Decision Tree model
model = DecisionTreeClassifier()
model.fit(X_train, y_train)

# 4. Predict using the test set
y_pred = model.predict(X_test)

# 5. Evaluate performance
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("Accuracy Score:", accuracy_score(y_test, y_pred))
```

Train-Test Split in Iris Dataset

When using machine learning, it's important to divide the data into two parts:

- **Training data:** used to train the model.
- **Testing data:** used to test how well the model performs on unseen data.

Variables Involved:

- **X** = Features (Input)
- **y** = Labels (Output / Target)
- **X_train** = Features for training the model
- **X_test** = Features for testing the model
- **y_train** = Labels used during training
- **y_test** = Actual labels used to test predictions

Iris Dataset Example with Explanation:

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split

# Load dataset
iris = load_iris()
X = iris.data # All feature columns (sepal/petal length/width)

y = iris.target # Labels (0 = Setosa, 1 = Versicolor, 2 = Virginica)

# Split into train and test sets (80% training, 20% testing) X_train,
X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

# Print the shapes to verify
print("X_train shape:", X_train.shape) # e.g., (120, 4)

print("X_test shape:", X_test.shape) # e.g., (30, 4)
print("y_train shape:", y_train.shape) # e.g., (120,)
print("y_test shape:", y_test.shape) # e.g., (30,)
```

Variable	Meaning
X_train	80% of feature data used for training
X_test	20% of feature data used for testing
y_train	Labels corresponding to X_train
y_test	Labels corresponding to X_test, used for evaluation